

```

    if insertSide == 'left':
        return '0' * numZeros + numberString
    elif insertSide == 'right':
        return numberString + '0' * numZeros

def karatsuba(x, y):
    """Multiply two integers with the Karatsuba algorithm. Note that
    the * operator isn't used anywhere in this function."""
    assert isinstance(x, int), 'x must be an integer'
    assert isinstance(y, int), 'y must be an integer'
    x = str(x)
    y = str(y)

    # At single digits, look up the products in the multiplication table:
    if len(x) == 1 and len(y) == 1: # BASE CASE
        print('Lookup', x, '*', y, '=', MULT_TABLE[(int(x), int(y))])
        return MULT_TABLE[(int(x), int(y))]

    # RECURSIVE CASE
    print('Multiplying', x, '*', y)

    # Pad with prepended zeros so that x and y are the same length:
    if len(x) < len(y): ❷
        # If x is shorter than y, pad x with zeros:
        x = padZeros(x, len(y) - len(x), 'left')
    elif len(y) < len(x):
        # If y is shorter than x, pad y with zeros:
        y = padZeros(y, len(x) - len(y), 'left')
    # At this point, x and y have the same length.

    halfOfDigits = math.floor(len(x) / 2) ❸

    # Split x into halves a & b, split y into halves c & d:
    a = int(x[:halfOfDigits])
    b = int(x[halfOfDigits:])
    c = int(y[:halfOfDigits])
    d = int(y[halfOfDigits:])

    # Make the recursive calls with these halves:
    step1Result = karatsuba(a, c) ❹ # Step 1: Multiply a & c.
    step2Result = karatsuba(b, d) # Step 2: Multiply b & d.
    step3Result = karatsuba(a + b, c + d) # Step 3: Multiply a + b & c + d.

    # Step 4: Calculate Step 3 - Step 2 - Step 1:
    step4Result = step3Result - step2Result - step1Result ❺

```